

2.2 话题 (Topic) 通信

所属阶段： 第二阶段 · ROS 2 核心通信机制

讲次数量： 8 节

预计时长： ~40 分钟

前置依赖： 2.1 ROS 2 节点 (Node) 基础

学习目标

完成本节后，学员将能够：

- 解释 ROS 2 Topic 的发布/订阅 (Pub/Sub) 通信模型及其与 DDS 的关系
- 使用 Python (rclpy) 编写 Publisher 和 Subscriber 节点
- 使用 C++ (rclcpp) 编写 Publisher 和 Subscriber 节点
- 运用 `ros2 topic` 系列命令行工具查看、调试和手动发布话题
- 通过 Turtlesim 实验验证 Topic 通信，编写节点控制海龟画圆
- 使用运行时重映射 (Remapping) 灵活调整话题名称，无需修改源码

讲次 1：什么是 ROS 2 Topic——发布/订阅模式的本质

知识点

从节点到话题：通信的必要性

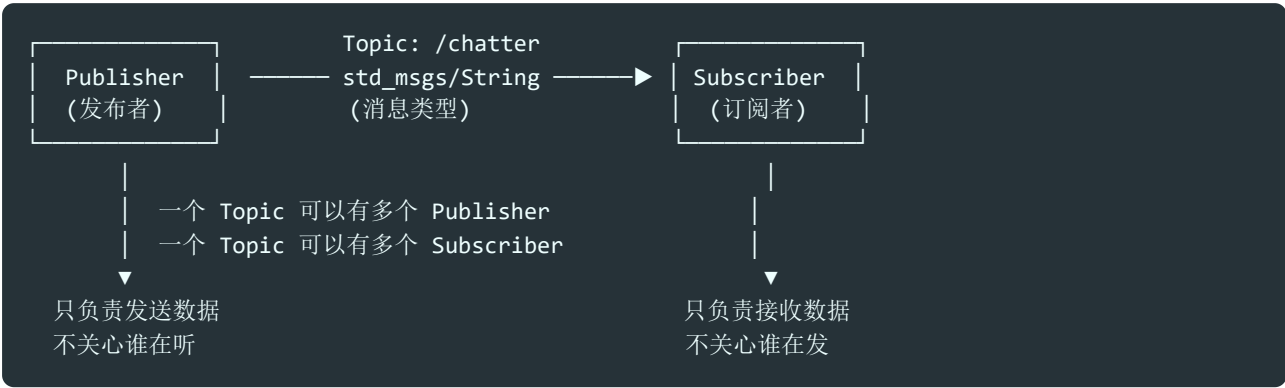
在 2.1 中，我们学习了 ROS 2 节点（Node）这一运行时逻辑单元。节点可以分布在不同进程或组合在同一进程，但单个节点通常不足以构成完整机器人系统，多个节点需要协同工作：

- 相机节点采集图像
- 感知节点处理图像并检测障碍物
- 规划节点生成运动路径
- 控制节点驱动电机执行运动

这些节点之间需要**交换数据**。ROS 2 提供了多种通信机制，其中 **Topic（话题）** 是最基础、使用最广泛的一种。

发布/订阅模式（Pub/Sub）

Topic 通信基于经典的**发布/订阅模式**：



核心特征：

特征	说明
异步	发布者发送后无需等待订阅者响应，继续执行
单向	数据从 Publisher 流向 Subscriber，无返回值
多对多	多个节点可同时发布或订阅同一话题
解耦	发布者与订阅者互不知晓对方的存在，只通过话题名和消息类型关联
持续流	适合传感器数据、状态信息等持续产生的数据流

Topic 通信的三个匹配维度

排查 Topic 通信时要同时检查三个维度：

1. **话题名称 (Topic Name)** : 字符串标识符, 通常使用小写加下划线, 如 `/chatter`、`/camera/image_raw`
2. **消息类型 (Message Type)** : 定义数据结构, 如 `std_msgs/msg/String`、`sensor_msgs/msg/Image`
3. **QoS 策略 (Quality of Service)** : 定义可靠性、历史深度、持久性等端点行为。QoS 不是 Topic 身份的一部分, 但发布端与订阅端必须兼容

话题名称的命名惯例:

```
/全局命名空间/topic_name
↑
以 / 开头表示全局命名空间, 所有节点都能访问
不以 / 开头则相对于节点自身的命名空间
```

Topic 在 ROS 2 架构中的位置

```
你的代码 (rclpy / rclcpp)
|
▼
create_publisher() / create_subscription()
|
▼
RCL (ROS Client Library)
|
▼
RMW (中间件抽象层)
|
▼
传输实现 (Jazzy 默认路径为 DDS)
|
▼
网络传输 / 共享内存
```

在 Fast DDS、Cyclone DDS 等 DDS 型 RMW 中, Publisher/Subscription 会映射为 DDS

DataWriter/DataReader, 中间件实现负责发现与数据传输。Zenoh 等非 DDS RMW 不使用这两个 DDS 实体, 因此这张分层图是课程默认环境的路径, 不是所有 RMW 的固定实现。

- **服务发现**: 自动匹配网络中的 Publisher 和 Subscriber
- **传输**: 具体实现可按配置使用 UDP、TCP 或共享内存等路径, 不能假定每条消息都走同一种传输

注意: 消息的序列化/反序列化由 **rosidl** 类型支持层和 RMW 完成, 而非 DDS 本身。

作为开发者, 你无需直接操作 DDS, 只需使用 rclpy/rclcpp 的高层 API。

Topic vs Service vs Action

ROS 2 提供三种主要通信机制, Topic 是最简单的一种:

通信机制	模式	方向	适用场景
Topic	Pub/Sub	单向、异步	传感器数据流、状态广播、连续控制指令
Service	请求/响应	双向；Client 可同步或异步等待	查询状态、触发一次性操作（如拍照）
Action	目标/反馈/结果	双向、异步	长时间任务（如导航到目标点）

选择 Topic 的典型场景：

- 激光雷达每秒发布 10 次扫描数据 → Topic
- 查询机器人当前电量 → Service
- 导航到指定位置（需要数分钟，途中需反馈进度）→ Action

消息类型体系

ROS 2 内置了大量标准消息类型，按功能分包：

包名	常用消息	用途
std_msgs	String, Int32, Float64, Bool	基础数据类型
geometry_msgs	Twist, Pose, Point	几何与运动学
sensor_msgs	Image, LaserScan, Imu	传感器数据
nav_msgs	Odometry, Path	导航相关
turtlesim	Pose, Color	Turtlesim 仿真（注意：不是 turtlesim_msgs）

消息类型的完整写法：

```
包名/msg/消息名
例：std_msgs/msg/String
    geometry_msgs/msg/Twist
```

在 Python 中导入： `from std_msgs.msg import String`

在 C++ 中导入： `#include "std_msgs/msg/string.hpp"`

讲次 2：【PY】编写 Publisher

知识点

创建功能包

Package 创建与构建流程见 2.1；本节新建 `py_pubsub`（依赖 `rclpy std_msgs`）：

```
cd ~/ros2_ws/src
ros2 pkg create --build-type ament_python py_pubsub --dependencies rclpy std_msgs
```

Publisher 的核心 API

API	作用
<code>Node.create_publisher(msg_type, topic_name, qos_depth)</code>	创建发布者
<code>Publisher.publish(msg)</code>	发布一条消息
<code>Node.create_timer(period_sec, callback)</code>	创建定时器，周期性触发回调

参数说明：

- `msg_type`：消息类型，如 `String`
- `topic_name`：话题名称，如 `"chatter"`（相对名称，实际为 `/chatter`）
- `qos_depth`：队列深度，表示最多缓存多少条未发送的消息，初学阶段设为 `10` 即可

完整代码：Publisher 节点

创建文件 `~/ros2_ws/src/py_pubsub/py_pubsub/publisher_node.py`：

```
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node
from std_msgs.msg import String

class PublisherNode(Node):
    """每秒向 /chatter 话题发布一条 String 消息的 Publisher 节点。"""

    def __init__(self):
        super().__init__("publisher_node")
        self.publisher_ = self.create_publisher(String, "chatter", 10)
        self.timer_ = self.create_timer(1.0, self.timer_callback)
        self.counter_ = 0
        self.get_logger().info("Publisher node started, publishing at 1 Hz")

    def timer_callback(self):
        msg = String()
        msg.data = f"Hello ROS 2! [{self.counter_}]"
        self.publisher_.publish(msg)
        self.get_logger().info(f'Publishing: "{msg.data}"')
        self.counter_ += 1

def main(args=None):
    rclpy.init(args=args)
    node = PublisherNode()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == "__main__":
    main()
```

代码说明

`create_publisher(String, "chatter", 10)` 创建发布者；`create_timer(1.0, ...)` 以 1 Hz 触发回调；在回调中构造 `String` 消息并 `publish()`。节点名与话题名相互独立。

注册入口点并构建

编辑 `setup.py`，添加入口点：

```
entry_points={
    'console_scripts': [
        'publisher_node = py_pubsub.publisher_node:main',
    ],
},
```

构建并运行：

```
cd ~/ros2_ws
colcon build --packages-select py_pubsub
source install/setup.bash
ros2 run py_pubsub publisher_node
```

预期输出：

```
[INFO] [publisher_node]: Publisher node started, publishing at 1 Hz
[INFO] [publisher_node]: Publishing: "Hello ROS 2! [0]"
[INFO] [publisher_node]: Publishing: "Hello ROS 2! [1]"
...
```

在另一个终端验证：

```
ros2 topic list          # 应看到 /chatter
ros2 topic echo /chatter # 实时查看消息内容
```

讲次 3：【PY】编写 Subscriber

知识点

Subscriber 的核心 API

API	作用
<code>Node.create_subscription(msg_type, topic_name, callback, qos_depth)</code>	创建订阅者

与 Publisher 不同，Subscriber 使用回调函数处理收到的消息。每当有新消息到达，ROS 2 自动调用回调函数。

完整代码：Subscriber 节点

创建文件 `~/ros2_ws/src/py_pubsub/py_pubsub/subscriber_node.py`：


```
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node
from std_msgs.msg import String

class SubscriberNode(Node):
    """订阅 /chatter 话题并打印收到的 String 消息。"""

    def __init__(self):
        super().__init__("subscriber_node")
        self.subscription_ = self.create_subscription(
            String,
            "chatter",
            self.listener_callback,
            10,
        )
        self.get_logger().info("Subscriber node started, listening on /chatter")

    def listener_callback(self, msg):
        self.get_logger().info(f'I heard: "{msg.data}"')

def main(args=None):
    rclpy.init(args=args)
    node = SubscriberNode()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == "__main__":
    main()
```

代码说明

`create_subscription(类型, 话题名, 回调, qos_depth)` —— 话题名和消息类型必须与 **Publisher** 一致。回调在 `spin()` 事件循环中被自动触发。

注册入口点并运行

在 `setup.py` 中添加入口点：

```
entry_points={
    'console_scripts': [
        'publisher_node = py_pubsub.publisher_node:main',
        'subscriber_node = py_pubsub.subscriber_node:main',
    ],
},
```

重新构建：

```
cd ~/ros2_ws
colcon build --packages-select py_pubsub
source install/setup.bash
```

终端 1 — 启动 Publisher：

```
ros2 run py_pubsub publisher_node
```

终端 2 — 启动 Subscriber：

```
ros2 run py_pubsub subscriber_node
```

预期输出（终端 2）：

```
[INFO] [subscriber_node]: Subscriber node started, listening on /chatter
[INFO] [subscriber_node]: I heard: "Hello ROS 2! [0]"
[INFO] [subscriber_node]: I heard: "Hello ROS 2! [1]"
...
```

通信匹配条件

Publisher 和 Subscriber 成功通信必须满足：

条件	说明
话题名一致	如都为 <code>chatter</code> 或 <code>/chatter</code>
消息类型一致	如都为 <code>std_msgs/msg/String</code>
QoS 兼容	重点核对 <code>reliability</code> 、 <code>durability</code> 等兼容关系；队列 <code>depth</code> 不要求数值相同
同一 ROS 域且网络可达	默认 Domain ID = 0；同机通常可达，跨机还受防火墙、组播和 RMW 配置影响

收不到消息时，按讲次 6 的排查顺序检查。

讲次 4: 【C++】编写 Publisher

知识点

创建 C++ 功能包

```
cd ~/ros2_ws/src  
ros2 pkg create --build-type ament_cmake cpp_pubsub --dependencies rclcpp std_msgs
```

完整代码: C++ Publisher 节点

创建文件 `~/ros2_ws/src/cpp_pubsub/src/publisher_node.cpp` :

```

#include <chrono>
#include <memory>
#include <string>

#include "rclcpp/rclcpp.hpp"
#include "std_msgs/msg/string.hpp"

using namespace std::chrono_literals;

class PublisherNode : public rclcpp::Node
{
public:
    PublisherNode()
    : Node("publisher_node"), counter_(0)
    {
        publisher_ = this->create_publisher<std_msgs::msg::String>("chatter", 10);
        timer_ = this->create_wall_timer(
            1s, std::bind(&PublisherNode::timer_callback, this));
        RCLCPP_INFO(this->get_logger(), "Publisher node started, publishing at 1 Hz");
    }

private:
    void timer_callback()
    {
        auto msg = std_msgs::msg::String();
        msg.data = "Hello ROS 2! [" + std::to_string(counter_) + "]";
        publisher_->publish(msg);
        RCLCPP_INFO(this->get_logger(), "Publishing: \"%s\"", msg.data.c_str());
        counter_++;
    }

    rclcpp::Publisher<std_msgs::msg::String>::SharedPtr publisher_;
    rclcpp::TimerBase::SharedPtr timer_;
    size_t counter_;
};

int main(int argc, char * argv[])
{
    rclcpp::init(argc, argv);
    auto node = std::make_shared<PublisherNode>();
    rclcpp::spin(node);
    rclcpp::shutdown();
    return 0;
}

```

代码说明

与 Python 对称：`create_publisher<std_msgs::msg::String>()` 创建发布者；`create_wall_timer(1s, std::bind(...))` 驱动 1 Hz 发布；消息头文件为全小写 `std_msgs/msg/string.hpp`。

构建并运行

```
cd ~/ros2_ws  
colcon build --packages-select cpp_pubsub  
source install/setup.bash  
ros2 run cpp_pubsub publisher_node
```

验证:

```
ros2 topic echo /chatter
```

讲次 5: 【C++】编写 Subscriber

知识点

完整代码: C++ Subscriber 节点

创建文件 `~/ros2_ws/src/cpp_pubsub/src/subscriber_node.cpp` :

```
#include <memory>
#include <string>

#include "rclcpp/rclcpp.hpp"
#include "std_msgs/msg/string.hpp"

class SubscriberNode : public rclcpp::Node
{
public:
    SubscriberNode()
    : Node("subscriber_node")
    {
        subscription_ = this->create_subscription<std_msgs::msg::String>(
            "chatter",
            10,
            std::bind(&SubscriberNode::listener_callback, this, std::placeholders::_1));
        RCLCPP_INFO(this->get_logger(), "Subscriber node started, listening on /chatter");
    }

private:
    void listener_callback(const std_msgs::msg::String::SharedPtr msg) const
    {
        RCLCPP_INFO(this->get_logger(), "I heard: \"%s\"", msg->data.c_str());
    }

    rclcpp::Subscription<std_msgs::msg::String>::SharedPtr subscription_;
};

int main(int argc, char * argv[])
{
    rclcpp::init(argc, argv);
    auto node = std::make_shared<SubscriberNode>();
    rclcpp::spin(node);
    rclcpp::shutdown();
    return 0;
}
```

代码说明

`create_subscription<MsgType>(topic, qos, std::bind(&Class::callback, this, _1))` —— 回调参数为 `const MsgType::SharedPtr` , 通过 `msg->data` 访问字段。

更新 CMakeLists.txt

在 CMakeLists.txt 中添加 subscriber 可执行文件：

```
add_executable(subscriber_node src/subscriber_node.cpp)
ament_target_dependencies(subscriber_node rclcpp std_msgs)

install(TARGETS
  publisher_node
  subscriber_node
  DESTINATION lib/${PROJECT_NAME}
)
```

构建并运行

```
cd ~/ros2_ws
colcon build --packages-select cpp_pubsub
source install/setup.bash
```

终端 1 — C++ Publisher:

```
ros2 run cpp_pubsub publisher_node
```

终端 2 — C++ Subscriber:

```
ros2 run cpp_pubsub subscriber_node
```

也可以交叉测试：Python Publisher + C++ Subscriber，或反之。这正是 ROS 2 的魅力——**语言无关的通信**。

交叉语言通信验证

终端 1（发布者）	终端 2（订阅者）	结果
ros2 run py_pubsub publisher_node	ros2 run cpp_pubsub subscriber_node	✓ 正常通信
ros2 run cpp_pubsub publisher_node	ros2 run py_pubsub subscriber_node	✓ 正常通信

消息类型和话题名一致即可，与编程语言无关。

讲次 6: Topic 命令行查看与调试

知识点

`ros2 topic` 是日常调试最高频的工具集。编写 Subscriber 前, 建议先用 `echo` 确认数据源。

命令	用途	常用选项
<code>ros2 topic list -t</code>	列出话题及类型	<code>-t</code> 显示消息类型
<code>ros2 topic info TOPIC</code>	发布者/订阅者数量	<code>-v</code> 查看 QoS 详情
<code>ros2 topic echo TOPIC</code>	实时打印消息	<code>--once</code> ; <code>--field x</code> 只看某字段
<code>ros2 topic hz TOPIC</code>	测量发布频率	
<code>ros2 topic pub TOPIC TYPE DATA</code>	手动发布	<code>--once</code> / <code>--rate N</code> / <code>--times N</code>
<code>ros2 topic bw TOPIC</code>	测量带宽	相机/点云等大数据话题

```
# 示例
ros2 topic list -t
ros2 topic echo /chatter --once
ros2 topic pub /chatter std_msgs/msg/String "{data: 'Hello from CLI'}" --once
ros2 interface show std_msgs/msg/String # 查看消息结构
```

通信问题排查顺序: `list -t` → `info` → `echo` → `hz` → `info -v` (查 QoS) 。

讲次 7: 【LAB】用 Turtlesim 实验 Topic 通信

知识点

Turtlesim 是 ROS 2 自带的轻量级 2D 仿真器，是学习 Topic 通信的最佳实验平台。本实验将手动操作和编程控制结合，全面体验 Topic 通信。

实验目标

1. 启动 Turtlesim 仿真器
2. 用命令行工具探索 Turtlesim 的话题
3. 手动发布速度指令控制海龟运动
4. 编写 Python 节点，控制海龟画圆

Step 1: 启动 Turtlesim

确保已安装 turtlesim 包（若已安装 `ros-jazzy-desktop`，可跳过此步，desktop 版已包含 turtlesim）：

```
sudo apt install ros-jazzy-turtlesim
```

终端 1 — 启动仿真器：

```
ros2 run turtlesim turtlesim_node
```

会弹出一个蓝色背景窗口，中央有一只海龟。

终端 2 — 启动键盘控制（可选，用于手动体验）：

```
ros2 run turtlesim turtle_teleop_key
```

在终端 2 中按方向键可以控制海龟移动（确保终端 2 处于焦点状态）。

Step 2: 用 `ros2 topic list` 查看话题

终端 3：

```
ros2 topic list -t
```

输出（核心话题）：

```
/parameter_events [rcl_interfaces/msg/ParameterEvent]  
/rosout [rcl_interfaces/msg/Log]  
/turtle1/cmd_vel [geometry_msgs/msg/Twist]  
/turtle1/color_sensor [turtlesim/msg/Color]  
/turtle1/pose [turtlesim/msg/Pose]
```

重点关注：

话题	类型	方向	说明
/turtle1/cmd_vel	geometry_msgs/msg/Twist	订阅	速度指令输入（我们发布到此话题）
/turtle1/pose	turtlesim/msg/Pose	发布	海龟位姿输出（位置 x,y 和朝向 theta）
/turtle1/color_sensor	turtlesim/msg/Color	发布	海龟当前颜色

Step 3: 用 ros2 topic echo 查看海龟位姿

```
ros2 topic echo /turtle1/pose
```

输出：

```
x: 5.544445037841797
y: 5.544445037841797
theta: 0.0
linear_velocity: 0.0
angular_velocity: 0.0
---
```

移动海龟（在终端 2 按方向键），观察 `x`、`y`、`theta` 的实时变化。

只查看 x 坐标：

```
ros2 topic echo /turtle1/pose --field x
```

查看消息结构：

```
ros2 interface show turtlesim/msg/Pose
```

```
float32 x
float32 y
float32 theta
float32 linear_velocity
float32 angular_velocity
---
```

Step 4: 用 ros2 topic pub 手动控制海龟

前进（线速度 0.5 m/s）：

```
ros2 topic pub /turtle1/cmd_vel geometry_msgs/msg/Twist \
  "{linear: {x: 0.5, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 0.0}}" --once
```

旋转（角速度 1.0 rad/s）：

```
ros2 topic pub /turtle1/cmd_vel geometry_msgs/msg/Twist \
  "{linear: {x: 0.0, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 1.0}}" --once
```

画圆运动（线速度 + 角速度同时施加）：

```
ros2 topic pub /turtle1/cmd_vel geometry_msgs/msg/Twist \
  "{linear: {x: 1.0, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 0.5}}" --rate 10
```

`--rate 10` 表示以 10 Hz 持续发布。观察海龟做圆周运动。按 `Ctrl+C` 停止。

理解 `geometry_msgs/msg/Twist` 的结构：

```
Vector3 linear    # 线速度 (x=前进, y=左右, z=上下)
  float64 x
  float64 y
  float64 z
Vector3 angular   # 角速度 (x=俯仰, y=翻滚, z=偏航)
  float64 x
  float64 y
  float64 z
```

对于平面运动的 Turtlesim，只需关注 `linear.x`（前进速度）和 `angular.z`（旋转速度）。

Step 5: 编写 Python 节点控制海龟画圆

创建功能包：

```
cd ~/ros2_ws/src
ros2 pkg create --build-type ament_python turtle_circle \
  --dependencies rclpy geometry_msgs
```

创建节点文件 `~/ros2_ws/src/turtle_circle/turtle_circle/circle_node.py`：

```
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Twist

class CircleNode(Node):
    """以恒定线速度和角速度控制海龟做圆周运动。"""

    def __init__(self):
        super().__init__("circle_node")
        self.publisher_ = self.create_publisher(Twist, "/turtle1/cmd_vel", 10)
        self.timer_ = self.create_timer(0.1, self.timer_callback) # 10 Hz
        self.get_logger().info(
            "Circle node started: linear.x=1.0, angular.z=0.5"
        )

    def timer_callback(self):
        msg = Twist()
        msg.linear.x = 1.0 # 前进速度 1.0 m/s
        msg.angular.z = 0.5 # 旋转速度 0.5 rad/s
        self.publisher_.publish(msg)

def main(args=None):
    rclpy.init(args=args)
    node = CircleNode()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        # 停止海龟运动
        if rclpy.ok():
            try:
                stop_msg = Twist()
                node.publisher_.publish(stop_msg)
            except Exception:
                pass
        node.destroy_node()
        if rclpy.ok():
            rclpy.shutdown()

if __name__ == "__main__":
    main()
```

注册入口点（编辑 `setup.py`）：

```
entry_points={
    'console_scripts': [
        'circle_node = turtle_circle.circle_node:main',
    ],
},
```

构建并运行：

```
cd ~/ros2_ws
colcon build --packages-select turtle_circle
source install/setup.bash
```

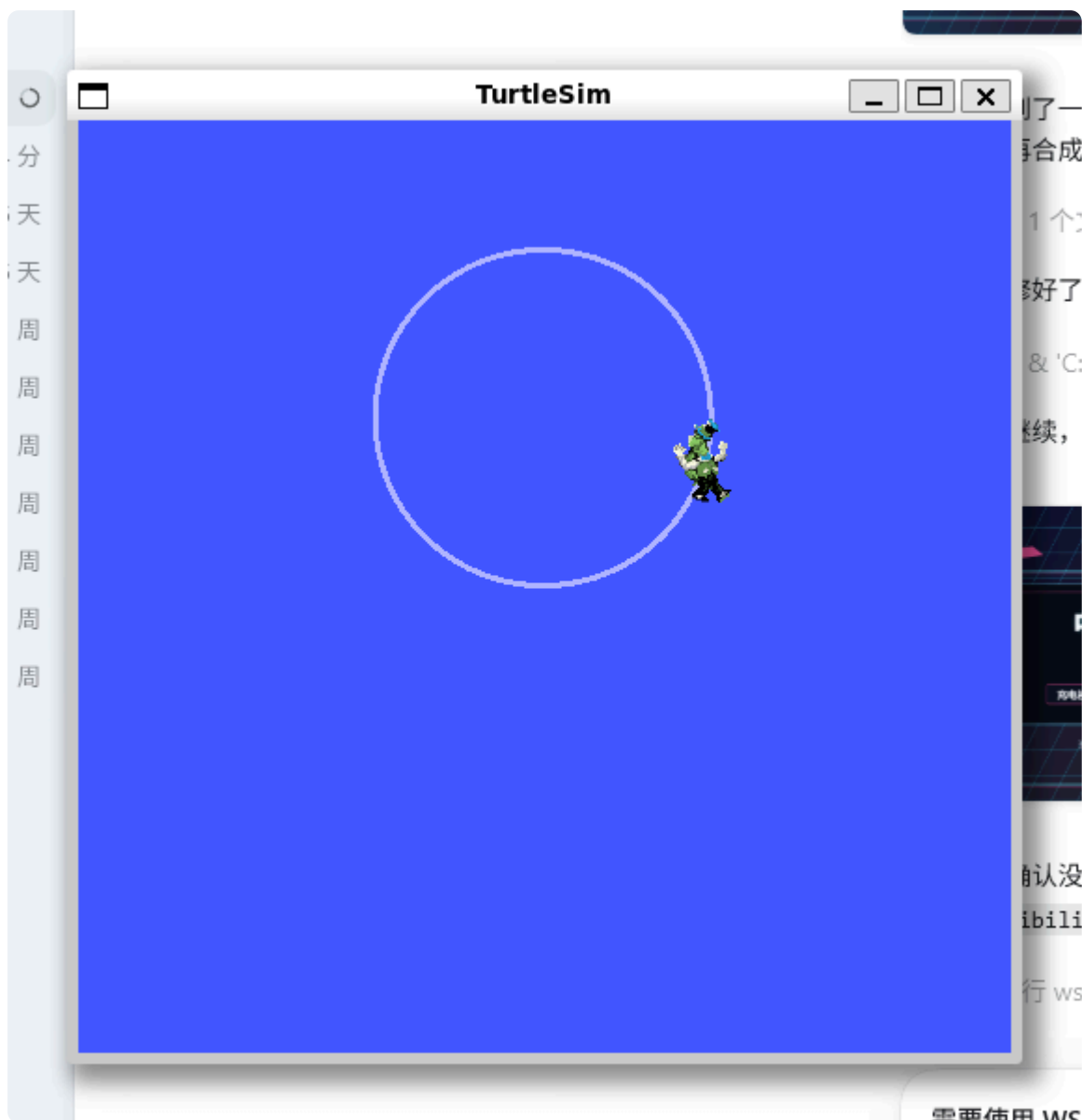
终端 1 — Turtlesim (保持运行)：

```
ros2 run turtlesim turtlesim_node
```

终端 2 — 画圆节点：

```
ros2 run turtle_circle circle_node
```

海龟开始做圆周运动。按 `Ctrl+C` 停止节点，海龟也会停止（`finally` 块发布零速度消息）。



上图为本次课件配套源码手动运行 `turtlesim_node` 与 `circle_node` 后截取的真实实验窗口画面。

终端 3 — 同时监控位姿和频率：

```
ros2 topic hz /turtle1/cmd_vel
ros2 topic echo /turtle1/pose --field theta
```

实验扩展（选做）

- 修改半径：**圆周半径 $R = v / \omega$ 。尝试 `linear.x=2.0, angular.z=1.0` ($R=2\text{m}$) 和 `linear.x=0.5, angular.z=1.0` ($R=0.5\text{m}$)，观察差异
- 订阅位姿：**在 `circle_node` 中添加对 `/turtle1/pose` 的订阅，每转一圈打印一次当前位置
- 正方形路径：**修改节点，让海龟走正方形（直线 2 秒 → 转弯 1.57 秒，重复 4 次）

常见错误

现象	原因	解决方案
海龟不动	<code>turtlesim_node</code> 未启动	先启动仿真器
海龟运动卡顿	发布频率太低	使用 10 Hz 以上（ <code>create_timer(0.1, ...)</code> ）
海龟走出边界	速度太大	降低 <code>linear.x</code> 或增大 <code>angular.z</code>
节点启动报错	未 source 工作空间	<code>source ~/ros2_ws/install/setup.bash</code>
键盘控制无效	终端未获得焦点	点击 <code>turtle_teleop_key</code> 所在终端

讲次 8：运行时 Topic 重映射

知识点

重映射 (Remapping) 允许在**运行时**修改节点的话题名，无需改源码或重编译——适合测试/仿真/多机器人场景切换。

语法

```
ros2 run <package> <node> --ros-args -r <原话题名>:=<新话题名>
```

可一次指定多个 `-r`。相对名、绝对名均可映射。

示例：Publisher 与 Subscriber 同步重映射

终端 1 — Publisher 发布到 `my_topic`：

```
ros2 run py_pubsub publisher_node --ros-args -r chatter:=my_topic
```

终端 2 — Subscriber 同样重映射（只改一端则无法通信）：

```
ros2 run py_pubsub subscriber_node --ros-args -r chatter:=my_topic
```

验证：`ros2 topic list -t` 应看到 `/my_topic`；`ros2 topic echo /my_topic` 有数据。

重映射 vs 修改源码

方式	优点	缺点
重映射	无需改代码/重编译，灵活切换	命令行较长
修改源码	话题名明确	不同场景需维护多份代码
Launch 文件	可版本管理、多节点统一配置	见 2.6 Launch 讲次

最佳实践：源码用通用话题名（如 `cmd_vel`、`scan`），部署时通过重映射或 Launch 适配。

其他重映射

```
# 节点名
ros2 run py_pubsub publisher_node --ros-args -r __node:=my_publisher

# 对接 Turtlesim: 将 cmd_vel 映射到仿真话题
ros2 run my_pkg controller --ros-args -r cmd_vel:=/turtle1/cmd_vel
```

Launch 文件中写法（预告）：`remappings=[('chatter', 'my_topic')]`。

本节总结

- **Topic 模型**: Pub/Sub 异步单向; 通信匹配需检查话题名、消息类型、QoS 兼容和 ROS Domain/网络可达性
- **编写节点**: `create_publisher` / `create_subscription` + 回调; Py/C++ API 对称, 可跨语言互通
- **调试**: `ros2 topic list/echo/hz/pub`; 排查顺序见讲次 6
- **重映射**: `--ros-args -r old:=new`, 通信双方须一致

命令速查表

命令	说明
<code>ros2 topic list</code>	列出所有活跃话题
<code>ros2 topic list -t</code>	列出话题及其消息类型
<code>ros2 topic info <topic></code>	查看话题的发布者/订阅者数量和类型
<code>ros2 topic info <topic> -v</code>	查看详细 QoS 和节点信息
<code>ros2 topic echo <topic></code>	实时打印话题消息内容
<code>ros2 topic echo <topic> --field <f></code>	只显示消息的指定字段
<code>ros2 topic echo <topic> --once</code>	只打印一条消息后退出
<code>ros2 topic hz <topic></code>	测量话题发布频率
<code>ros2 topic bw <topic></code>	测量话题网络带宽
<code>ros2 topic pub <topic> <type> <data></code>	手动发布消息
<code>ros2 topic pub <topic> <type> <data> --once</code>	只发布一次
<code>ros2 topic pub <topic> <type> <data> --rate 10</code>	以 10 Hz 持续发布
<code>ros2 interface show <type></code>	查看消息类型定义
<code>ros2 run pkg node --ros-args -r old:=new</code>	运行时话题重映射
<code>ros2 run turtlesim turtlesim_node</code>	启动 Turtlesim 仿真器
<code>ros2 run py_pubsub publisher_node</code>	运行 Python Publisher
<code>ros2 run py_pubsub subscriber_node</code>	运行 Python Subscriber
<code>ros2 run cpp_pubsub publisher_node</code>	运行 C++ Publisher
<code>ros2 run cpp_pubsub subscriber_node</code>	运行 C++ Subscriber
<code>ros2 run turtle_circle circle_node</code>	运行海龟画圆节点

核心 API 速查

操作	Python (rclpy)	C++ (rclcpp)
创建 Publisher	<code>create_publisher(Type, "topic", 10)</code>	<code>create_publisher<Type>("topic", 10)</code>
创建 Subscriber	<code>create_subscription(Type, "topic", cb, 10)</code>	<code>create_subscription<Type>("topic", 10, cb)</code>
发布消息	<code>publisher_.publish(msg)</code>	<code>publisher_->publish(msg)</code>
定时器	<code>create_timer(1.0, callback)</code>	<code>create_wall_timer(1s, std::bind(...))</code>
事件循环	<code>rclpy.spin(node)</code>	<code>rclcpp::spin(node)</code>

下一步

下一节 **2.3 服务 (Service) 通信** 中，我们将学习 ROS 2 的第二种通信机制——Service。与 Topic 的异步单向数据流不同，Service 提供**同步的请求-响应**模式，适合查询状态、触发一次性操作等场景。我们将编写 Python 和 C++ 的 Service Server 与 Client，并用 Turtlesim 的 `/spawn`、`/turtle1/set_pen` 等服务完成实验。